T-GAPE: Does the Leaf Level Matter? Topology-Aware Rank Mapping for ML Traffic on Oversubscribed Leaf-Spine Clusters

Divyansh Verma, Aniruddha Badhani, Ashutosh Singh
Dept. of Computer Science, Noida Institute of Engineering and Technology, Greater Noida, India
{0241cs202, 0241cs161, 0241cs203}@niet.co.in

***Abstract*—Communication-aware placement usually stops at the host: it decides which ranks share a machine and leaves the mapping of hosts onto switches to chance. We ask how much the next level, which hosts share a leaf switch, matters for modern training traffic, and under what spine oversubscription. Using a two-tier leaf-spine model, three workloads (MoE with routing affinity, Megatron with mismatched rank order, and a 3D stencil) and four mapping strategies, we find that the leaf level adds nothing at 1:1 oversubscription, even though leaf-aware mapping cuts cross-leaf bytes from 0.55 to 0.31 of traffic for MoE. It matters once the host-only mapping becomes uplink-bound: at 4:1, hierarchical mapping reaches 0.40× block placement's modelled communication time for MoE against 0.74× for host-only mapping, and 0.07× against 0.14× for Megatron. Under NCCL-style ring re-derivation the gaps shrink but hold (0.42× vs. 0.67×; 0.11× vs. 0.18×). For the stencil, bottom-up mapping is 1.02× block while top-down reaches 0.79×: minimising total cut does not minimise the busiest uplink. All results are scenario values from an analytic model.**

***Index Terms*—topology mapping, leaf-spine, oversubscription, process placement, distributed training**

I. Introduction

Hierarchical topology mapping is well studied for HPC: Hoefler and Snir compare generic mapping strategies [1], TreeMatch maps onto machine trees [2], and Scotch recursively bipartitions process and architecture graphs together [3]; see [4] for a survey. We do not claim that two-level mapping is new. The open question is quantitative: *for the traffic of modern training jobs, how much does mapping hosts onto leaf switches add to host-level placement, and when?* The answer decides whether a placement tool for GPU clusters needs a topology model at all.

Our companion paper [5] (CAGE) shows that host-level placement beats block placement only when the communication graph has significant structure that disagrees with rank order. This paper takes those cases, adds a leaf-spine fabric, and measures the increment. Our contributions:

1) A leaf-spine extension of the CAGE cost model with four mapping strategies, evaluated in both the rank-pinned and the NCCL ring re-derivation regimes (Section III).
2) Evidence that the leaf level matters exactly when host-only mapping is uplink-bound: never at 1:1, for some workloads at 2:1, and strongly at 4:1 (Section IV).
3) A counter-example in which bottom-up mapping is worse than block placement, traced to a mismatch between the min-cut objective and the bottleneck-link cost.

II. Background

GPU clusters are commonly built as folded-Clos (fat-tree) fabrics [6], [7]. Each host's NIC connects to a leaf switch; leaves connect to spines. Operators often provision less uplink bandwidth than the sum of host links, a ratio called oversubscription [8]. Traffic between hosts on one leaf never touches the uplinks; traffic between leaves does, spread over spines by ECMP [9]. TopoOpt co-optimises topology and parallelisation for ML jobs [10]; FasterMoE uses topology in MoE expert placement [11]. We take the fabric as given and change only where ranks run.

III. Model and Strategies

Fabric. Λ leaves, $k = 4$ hosts per leaf, $g = 8$ GPUs per host. Each host has one NIC of $B_{\text{nic}} = 50$ GB/s per direction; GPUs in a host share $B_{\text{intra}} = 300$ GB/s. Each leaf has one aggregated uplink of $B_{\text{up}} = kB_{\text{nic}}/\rho$ for oversubscription $\rho \in \{1, 2, 4\}$. The spine is an ideal non-blocking crossbar (perfect ECMP).

Cost. Given the rank traffic matrix M, the host placement and the leaf of each host, let N_h be host h's larger direction of cross-host bytes, U_ℓ leaf ℓ's larger direction of cross-leaf bytes, and X_h host h's intra-host bytes. The modelled per-step communication time is

$$T = \max\left(\max_h \frac{N_h}{B_{\text{nic}}},\ \max_\ell \frac{U_\ell}{B_{\text{up}}}\right) + \max_h \frac{X_h}{B_{\text{intra}}}. \tag{1}$$

A mapping is *uplink-bound* when its busiest uplink time exceeds its busiest NIC time. Like CAGE's host model, T ranks mappings; it does not predict seconds.

Strategies. All use CAGE's exact-capacity partitioner (METIS, capacity repair, pairwise swap refinement, best of three seeds):

- *block*: ranks in order onto hosts, hosts in order onto leaves (launcher default);
- *flat*: CAGE host partition with host labels matched to block placement, hosts onto leaves in that order (host-aware, leaf-oblivious);
- *hierarchical* (bottom-up): CAGE host partition, then partition the host traffic graph onto leaves;

TABLE I
MODELLED TIME RELATIVE TO BLOCK, AND WHETHER THE MAPPING IS UPLINK-BOUND (U). LAST COLUMN: SHARE OF TRAFFIC CROSSING LEAVES, FLAT → HIERARCHICAL (T1).

Workload	ρ	flat	hierarchical	top-down	cross-leaf
MoE, pinned	1	0.498	0.498	0.504	0.546 → 0.313
	2	0.605 U	0.498	0.504	
	4	0.736 U	0.398 U	0.398 U	
MoE, NCCL	1	0.525	0.525	0.531	0.379 → 0.221
	4	0.674 U	0.419 U	0.419 U	
Megatron, fixed	1	0.228	0.228	0.228	0.058 → 0.003
	4	0.140 U	0.071	0.071	
Megatron, NCCL	1	0.219	0.219	0.219	0.048 → 0.003
	4	0.182 U	0.109	0.109	
Stencil 3D	1	0.857	0.857	0.857	0.158 → 0.158
	4	1.019 U	1.019 U	0.792 U	

- *top-down*: partition ranks onto leaves ($kg = 32$ slots each), then onto hosts within each leaf.

Regimes. In the *pinned/fixed* regime the traffic matrix moves with the ranks. In the *NCCL* regime each recorded collective is re-expanded for every candidate as a ring ordered by leaf and then by host, as NCCL derives rings from the topology; each strategy takes the better of two partitioning objectives (all traffic in rank order, or rank-pinned traffic plus a clique expansion of the collectives), as in CAGE.

Workloads (128 ranks, 16 hosts, 4 leaves). These are the cases where host placement pays [5]: (i) *MoE*, 128-way expert parallelism, top-2, Zipf 1.0 popularity, data groups interleaved across ranks with routing affinity 0.6, plus a dense 2 GB allreduce (dropped in the pinned regime, re-ringed in the NCCL regime); (ii) *Megatron* TP 8, PP 4, DP 4 with randomly relabelled ranks; (iii) a *3D stencil* on an 8×4×4 grid with 4 MB halos, which is all point-to-point, so both regimes coincide.

IV. RESULTS

Fig. 1 plots modelled time relative to block for every workload and strategy; Table I lists the key rows.

A. At 1:1 the leaf level adds nothing

At $\rho = 1$, flat and hierarchical mapping give the same modelled time for every workload (MoE pinned 0.498, Megatron 0.228, stencil 0.857), although hierarchical mapping cuts the share of traffic crossing leaves from 0.546 to 0.313 for MoE and from 0.058 to 0.003 for Megatron. No mapping is uplink-bound: the busiest NIC is the bottleneck, and the NIC load depends only on the host partition. Cross-leaf bytes, the metric topology-mapping papers often report, are the wrong target when the uplinks are not the bottleneck.

B. Under oversubscription it is decisive

At $\rho = 2$, flat MoE mapping becomes uplink-bound (busiest uplink 1728 ms vs. busiest NIC 1402 ms) and rises to 0.605× block, while hierarchical mapping stays NIC-bound at 0.498×. At $\rho = 4$ the gap widens: 0.398× vs. 0.736× for MoE and 0.071× vs. 0.140× for Megatron. Hierarchical Megatron mapping never becomes uplink-bound: its busiest uplink carries 86 ms against a 408 ms NIC. With the leaf level, Megatron's time is the same at every oversubscription (1.21 s per step); without it, the flat mapping's time grows from 1.21 s to 2.38 s. The rule is simple and can be checked before anything moves: *map onto leaves when the host-only mapping is uplink-bound*.

C. NCCL re-ringing narrows but keeps the gap

In the NCCL regime the dense MoE allreduce and Megatron's TP and DP rings are rebuilt for each mapping in leaf-then-host order, which removes part of the cross-leaf ring traffic even for leaf-oblivious mappings. At $\rho = 4$ the gap shrinks from 0.398 vs. 0.736 to 0.419 vs. 0.674 for MoE, and from 0.071 vs. 0.140 to 0.109 vs. 0.182 for Megatron. The ratio of flat to hierarchical time falls from 1.97 to 1.67 for Megatron, but leaf-aware mapping still wins by a wide margin. Leaving the ring order to the library is not a substitute for placing hosts.

D. Min-cut is not min-max: the stencil

For the 3D stencil at $\rho = 4$, flat and hierarchical mapping are *worse* than block (1.019×), while top-down reaches 0.792×. All four strategies send the same share of traffic across leaves (0.158). What differs is how evenly it is spread: block, flat and hierarchical leave the busiest uplink at 2.56 ms, whereas top-down balances it to 1.92 ms. Bottom-up mapping fixes host shapes that minimise the host cut and then cannot arrange them onto leaves any better than block order does; the slight loss comes from the larger intra-host term of the partitioned hosts. The partitioners minimise *total* cut while the fabric is limited by its *busiest* link. On the ML workloads top-down and hierarchical agree (MoE 0.398 and 0.398, Megatron 0.071 and 0.071), so for them a two-level greedy mapping suffices; for regular HPC grids the choice matters.

V. THREATS TO VALIDITY

Synthetic inputs. All workloads are models, chosen from the cases where host placement pays. Where host placement does not pay (data parallelism, Megatron in launcher order), we have not tested whether leaf placement does. **Fabric model.** One aggregated uplink per leaf with ideal ECMP hides flow collisions on individual spine links, which in practice make oversubscription worse. Three-tier fat-trees, rail-optimised fabrics with one NIC per GPU, and dragonflies [12] are not modelled. **Single configuration.** Only 4 hosts per leaf, 16 hosts and 128 ranks were run. **Cost model.** Bandwidth only; no latency, compute overlap or dependencies. Times rank mappings; they do not predict seconds. **Validation path.** A flow- or packet-level simulator such as ASTRA-sim's network backend [13] or SimGrid [14], and eventually a real cluster, should replay these mappings.

VI. CONCLUSION

For modern training traffic on leaf-spine clusters, the leaf level is irrelevant at 1:1 oversubscription and decisive at 4:1, and the switch between the two is visible in the model: it happens when the host-only mapping becomes uplink-bound.

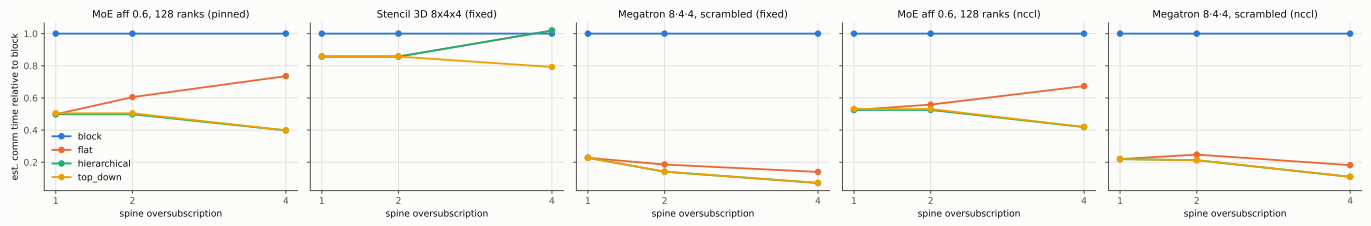


Fig. 1. T1: modelled communication time relative to block placement against spine oversubscription, per workload and regime. Scenario values.

NCCL's ring re-derivation narrows the gap but does not close it. Bottom-up two-level mapping can lose to block placement on regular grids because it minimises total cut rather than the busiest link; a min-max objective at the leaf level is the natural fix and the next experiment.

Artifact Availability

Code, the T1 script and its results are available at https://github.com/n0tdivyansh/cage-research; `cage run --paper 2` regenerates them.

References

[1] T. Hoefler and M. Snir, "Generic topology mapping strategies for large-scale parallel architectures," in *Proceedings of the International Conference on Supercomputing (ICS)*, 2011, pp. 75–84.

[2] E. Jeannot, G. Mercier, and F. Tessier, "Process placement in multicore clusters: Algorithmic issues and practical techniques," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 4, pp. 993–1002, 2014.

[3] F. Pellegrini and J. Roman, "SCOTCH: A software package for static mapping by dual recursive bipartitioning of process and architecture graphs," in *High-Performance Computing and Networking (HPCN Europe)*, ser. LNCS, vol. 1067, 1996, pp. 493–498.

[4] T. Hoefler, E. Jeannot, and G. Mercier, "An overview of topology mapping algorithms and techniques in high-performance computing," in *High-Performance Computing on Complex Environments*. Wiley, 2014, pp. 73–94.

[5] D. Verma, A. Badhani, and A. Singh, "CAGE: When does communication-graph structure pay off for rank placement?" https://github.com/n0tdivyansh/cage-research, 2026, companion paper (Paper 1 of this series).

[6] C. E. Leiserson, "Fat-trees: Universal networks for hardware-efficient supercomputing," *IEEE Transactions on Computers*, vol. C-34, no. 10, pp. 892–901, 1985.

[7] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," in *Proceedings of ACM SIGCOMM*, 2008, pp. 63–74.

[8] A. Greenberg *et al.*, "VL2: A scalable and flexible data center network," in *Proceedings of ACM SIGCOMM*, 2009, pp. 51–62.

[9] C. E. Hopps, "Analysis of an equal-cost multi-path algorithm," RFC 2992, 2000.

[10] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, "TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

[11] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li, "FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2022, pp. 120–134.

[12] J. Kim, W. J. Dally, S. Scott, and D. Abts, "Technology-driven, highly-scalable dragonfly topology," in *Proceedings of the 35th International Symposium on Computer Architecture (ISCA)*, 2008, pp. 77–88.

[13] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-SIM: Enabling SW/HW co-design exploration for distributed DL training platforms," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020.

[14] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, "Versatile, scalable, and accurate simulation of distributed applications and platforms," *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014.